



Master 1 (M1)

*Département d'informatique
l'UFR-SEA/
UJKZ*

Bases de Données Avancées

*Dr Kiswendsida Kisito Kaboré ,
Enseignant Chercheur Au
Département d'Informatique
à l'UFR-SEA / Université JKZ*

Janvier 2023

<http://kisitokab.xyz>



- **Prérequis**
 - Principes des systèmes de base de données
- **Volume horaire**
 - 30 heures
 - 3 Crédits
- **Méthode d'évaluation**
 - 1 Devoir
 - 1 Projet



Justification du Cours/ module



Résultats de l'apprentissage



Contenu-sujets du Cours

- Introduction à la conception des BdD
- Introduction à la réalisation des BdD
- Introduction à l'administration des BdD
- Les BdD réparties
- Les BdD Objets
- Les extensions XML



Master 1 (M1)

*Département d'informatique
l'UFR-SEA/
UJKZ*

CHAPITRE 3

Introduction à l'administration des Bases de Données

*Dr Kiswendsida Kisito Kaboré ,
Enseignant Chercheur Au
Département d'Informatique
à l'UFR-SEA / Université JKZ*

Janvier 2023

<http://kisitokab.xyz>

Rappel sur l'Administration d'une BdD :

- *Tâches d'administration*
- *PL/SQL+Triggers*
- *Indexation,*
- *Amélioration des performances*
et
- *Traitement et optimisation des requêtes*

•Objectif 1' :

Administration :

Les métiers autour des bases de données

- Administrateur
- Responsable de la sécurité
- Administrateur réseaux
- Développeurs d'application
- Administrateurs d'application
- Utilisateurs : modifier les données, créer des rapports

Note

Dans des environnements de petite taille, l'administrateur peut jouer quasiment tous les rôles

- .

Rôles du DBA

- Installer les logiciels Oracle
 - ✓ un serveur, des applications clientes,
 - ✓ En fonction de la configuration système disponible
 - ✓ Si fonctionnement en réseau : composants réseaux d'Oracle
- Planifier et créer des bases de données
- Gérer l'espace et implanter les schémas des données
- Assurer la sécurité, l'intégrité et la pérennité des données
- Effectuer des réglages pour optimiser les performances

•Objectif 2' :

Indexation :

- **Indexation :**

- Les index sont utilisés pour accélérer l'accès aux relations. Une relation ayant un index sur un attribut A permettra de retrouver rapidement les tuples ayant une valeur donnée pour cet attribut.

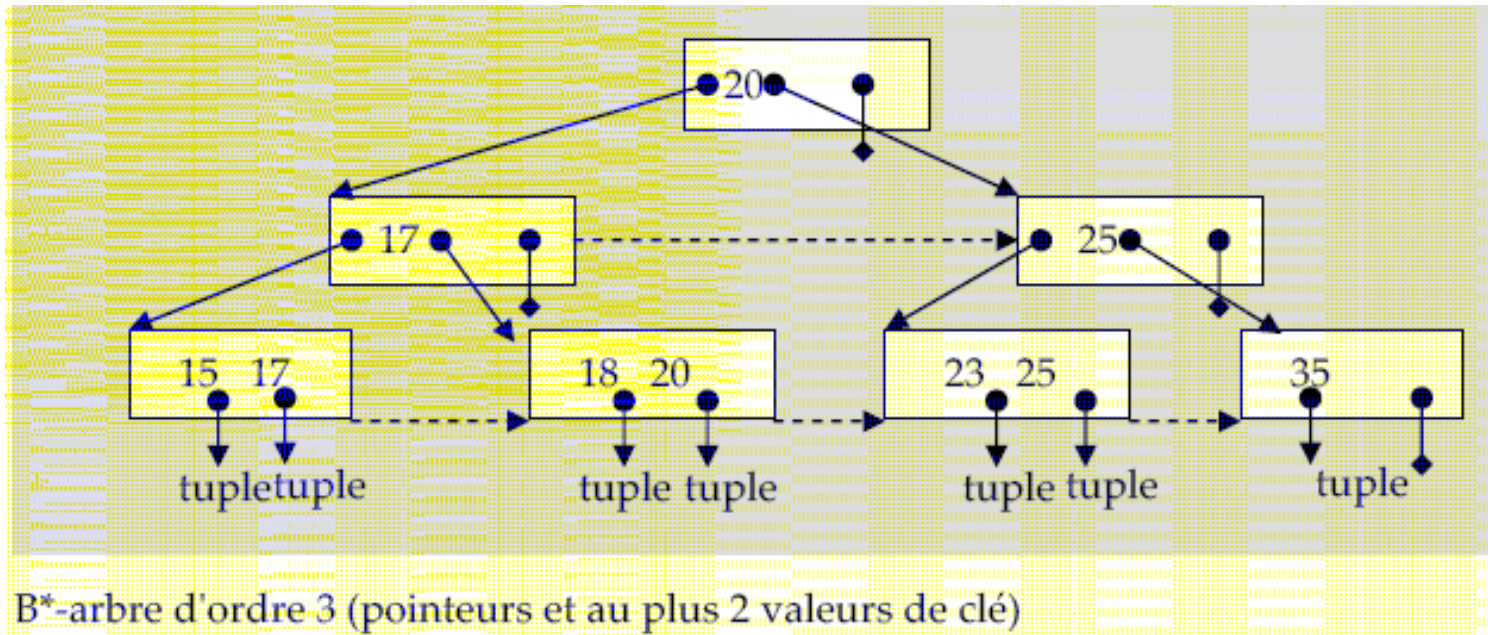
- **On a les index**

- primaires
 - de cluster
 - secondaire
- Utilise pour cela un fichier organisé en B-tree ou en B*-tree
- Cf Gamache.

- Structures de données :

- Des structures de données comme les arbres binaires et les clés de Hachage sont utilisées pour faciliter la recherche et le stockage dans les fichiers.

- Indexation :



- Fig 01

Création d'un index secondaire

Il y a 2 types d'index secondaires :

Unique : aucune valeur ne se répète,

Non unique : peut contenir des doublons

Syntaxe :

Create [unique] index Nom_Index

On Nom_Table (colonne1, ... colonneN);

Notes :

Un index sur le champs Prénom est différent d'un index sur les champs Prénom et Nom,

Un index sur les champ Prénom et Nom est différent d'un index sur les champs Nom et Prénom.

Suppression d'un index secondaire

Syntaxe :

Drop index Nom_Index ;

Objectif-3'

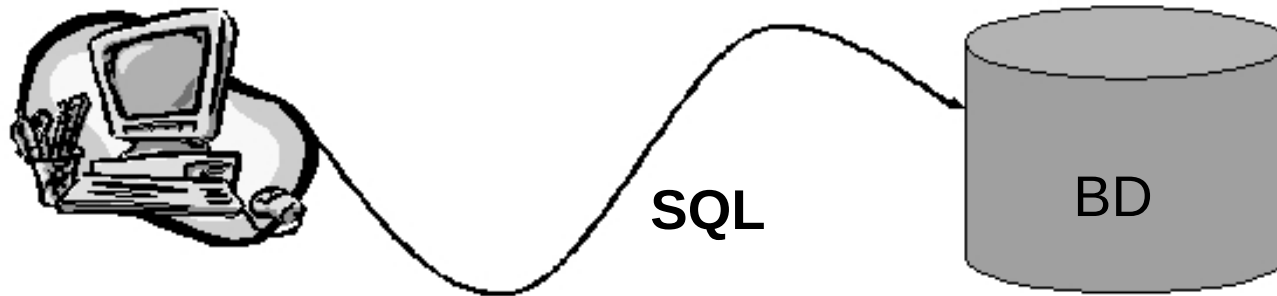
Exemple de langage procédurale :
Le langage PL/SQL

Oracle PL/SQL:

Objectifs

- „ Introduction
- „ Structure d'un programme PL/SQL
- „ Structures de contrôle
- „ Gestion des exceptions
- „ PL/SQL + SQL
- „ Curseurs
- „ Procédures et fonctions
- „ Triggers.

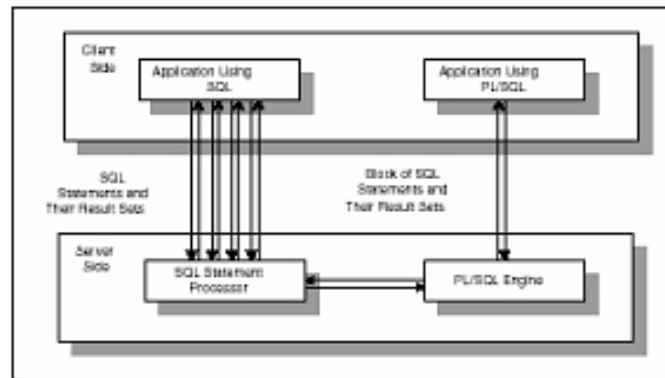
Introduction PL/SQL



- „ SQL permet de chercher les données dans la BD,
- „ SQL reste toutefois laborieux pour un utilisateur final,
- „ Besoin d'un langage de programmation pour effectuer des traitements sur les données. PL/SQL: Procedural Language Extensions to SQL,
- „ Solution propre à Oracle (chaque SGBD a le sien),
- „ Apporter des capacités de programmation à SQL,
- „ Définition des triggers.

Introduction PL/SQL

- „ Plusieurs requêtes SQL : trafic sur le réseau
- „ Un seul bloc PL/SQL:
 - ... Contient plusieurs requêtes SQL,
 - ... Exécution au niveau serveur car stocké sur le serveur,
 - ... Diminuer le trafic sur le réseau,



Structure d'un programme PL/SQL

DECLARE

section déclaration variable

BEGIN

section exécution du code

EXCEPTION

section Gestion d'exceptions

END ;

Structure d'un programme PL/SQL

Exemple:

DECLARE

```
V_PRENOM varchar2(35);  
V_NOM varchar2(35);  
V_COMPTEUR:= 0;
```

BEGIN

```
select PRENOM, NOM into V_PRENOM, V_NOM  
from ETUDIANT  
Where ETUDIANT_ID = 123;
```

```
DBMS_OUTPUT.PUT_LINE ('nom etudiant :'|| V_PRENOM  
|| ' ' || V_NOM);
```

EXCEPTION

```
WHEN NO DATA FOUND THEN
```

```
DBMS_OUTPUT.PUT_LINE
```

```
('Il n'y a aucun étudiant avec id 123');
```

END;

Déclaration et types de données

```
nomVariable [CONSTANT] data_type  
            [NOT NULL]  
            [:= (or DEFAULT) value];
```

„ La portée d'une variable est liée au bloc où elle est déclarée,

„ **Exemples:**

```
num NUMBER(2);  
nextTaxFilingDate CONSTANT DATE := '15-APR-00';  
vNom etudiant.nom%TYPE;
```

„ **vNom** est de **même type** que le champ **nom** de la table **etudiant**.

Déclaration et types de données

Type de données de bases SQL(rappel):

- ... **NUMBER** (précision, virgule)ou NUMBER(précision) ou NUMBER,
- ... **CHAR** (taille), -Longueur fixe,
- ... **VARCHAR** (taille), -moins de 2000 caractères,
- ... **VARCHAR2** (taille),-moins de 4000 caractères,
- ... **DATE**,

Type de données de bases PL/SQL :

- ... Les types SQL et:
- ... **BOOLEAN** : True ou False,
- ... **BINARY_INTEGER**: Entiers signés,
- ... **%TYPE** Même type que le champ mentionné.

Opérateurs PL/SQL

- „ Arithmétique : **, *, /, +, -
- „ Comparaison : =, <>, !=, <, >, <=, >=, LIKE, IN, BETWEEN, Is, NULL
- „ logiques: AND, OR, NOT
- „ Chaînes de caractères: || , LIKE

Structures de contrôle (conditionnelles)

```
IF boolean_exp1 THEN Instruction;  
[ELSIF boolean_exp2 THEN  
Statement2;]  
...  
[ELSE statementN;]  
ENDIF;
```

Structures de contrôle (conditionnelles)

Exemples

```
IF (score >= 90) THEN
    na := na +1;
ELSIF (score >= 80) THEN
    nb := nb +1 ;
ELSIF (score >= 70) THEN
    nc := nc + 1;
ELSE
    nd := nd + 1;
ENDIF;
```

Structures de contrôle (conditionnelles)

```
DECLARE
```

```
    vDate DATE := TO_DATE(SYSDATE, 'DD-MON-YYYY');
```

```
    vDay VARCHAR2(15);
```

```
BEGIN
```

```
    vDay := TO_CHAR(vDate, 'DAY');
```

```
    IF vDay IN ('Samedi', 'Dimanche')
```

```
    THEN
```

```
        DBMS_OUTPUT.PUT_LINE(vDate||' est dans la fin  
        de semaine');
```

```
    ENDIF
```

```
    DBMS_OUTPUT.PUT_LINE('Done...');
```

```
END;
```

Structures de contrôle (boucles)

LOOP

[EXIT [WHEN condition]];

END LOOP;

LOOP

i := i + 1;

EXIT WHEN i>10;

sum := sum + i;

END LOOP;

Structures de contrôle (boucles)

```
WHILE condition
    LOOP
        Statements;
    END LOOP;
```

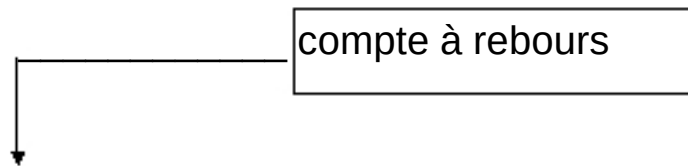
```
WHILE (i <= 10)
    LOOP
        dbms_output.put_line(i);
        i := i+1;
    END LOOP;
```

Structures de contrôle (boucles)

```
FOR loop_index IN [REVERSE] borne_inf .. Borne_sup  
  LOOP  
    Statements;  
  END LOOP;
```

```
FOR i IN 1..10  
  LOOP  
    dbms_output.putline ('i= ' || i);  
    sum := sum + i;  
  END LOOP;
```


Structures de contrôle (boucles)



```
FOR i IN REVERSE 1..10
```

```
  LOOP
```

```
    dbms_output.putline('i= '|| i);
```

```
    sum := sum + i;
```

```
  END LOOP;
```

Gestion des exceptions

- „ Dans PL/SQL, les conditions d'erreur du système sont appelées exceptions.
- „ Section de gestion des exceptions est optionnelle dans PL/SQL.
- „ Au lieu de vérifier les erreurs après chaque instruction, on indique comment gérer certaines conditions particulières.
- „ Il y a deux catégories:
 - ... prédéfinies,
 - ... Définies par le programmeur.

Gestion des exceptions

```
DECLARE
    vName VARCHAR2(30);
BEGIN
    SELECT firstName||' '|| lastName
    INTO vName FROM student
    WHERE studentId = 123456;
    DBMS_OUTPUT.PUT_LINE('Le nom de l''étudiant est '||vName);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Il n''y a pas
            d''étudiant');
END;
```

Gestion des exceptions

Exception globale et locale:

Declare

.....

Begin

.....

Begin

.....

EXCEPTION _____ Exception locale

.....

END ;

EXCEPTION _____ Exception globale

.....

END ;

Gestion des exceptions

```
DECLARE
```

```
    vName VARCHAR2(30);
```

```
    vRegistered CHAR;
```

```
BEGIN
```

```
    SELECT firstName||' '|| lastName
```

```
    INTO vName
```

```
    FROM student
```

```
    WHERE studentId = 12;
```

```
    DBMS_OUTPUT.PUT_LINE('Le nom de l''étudiant est ' ||vName);
```

```
BEGIN
```

```
    SELECT 'Y'
```

```
    INTO vRegistered
```

```
    FROM enrollment
```

```
    WHERE studentId = 12;
```

```
    DBMS_OUTPUT.PUT_LINE('L''étudiant est inscrit');
```

20

Gestion des exceptions

```
EXCEPTION
WHEN VALUE_ERROR OR INVALID_NUMBER THEN
    DBMS_OUTPUT.PUT_LINE('Une erreur est
survenue');
END
```

← Exception locale


```
EXCEPTION
WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Cet étudiant n''existe pas');
END;
```

← Exception globale

Gestion des exceptions

„ Exceptions prédéfinies

... **INVALID_NUMBER** : Une valeur numérique est invalide
(fonction TO_NUMBER par exemple),

... **DUP_VAL_IN_INDEX** : On tente de créer une valeur
dupliquée dans une colonne refusant les doublons,

... **NO_DATA_FOUND** : Un SELECT ne retourne rien,

... **TOO_MANY_ROWS** : Le SELECT a retourné plus d'une
ligne,

... **VALUE_ERROR** : Dans des situations d'erreur résultant de
valeurs tronquées ou converties (trop de caractères
dans un VARCHAR2(5)).

Gestion des exceptions

- „ Exception définies par le programmeur,
- „ Déclaration (dans la partie déclarative):

```
DECLARE
    eExceptionNom EXCEPTION;
BEGIN
    .....
EXCEPTION
    WHEN eExceptionNom
    THEN .....
END ;
```


Gestion des exceptions

„ Une exception utilisateur doit être déclenchée explicitement

```
DECLARE
eExceptionNom Exception;
BEGIN
...
IF CONDITION THEN RAISE eExceptionNom;
ELSE
... END IF;
EXCEPTION
WHEN eExceptionNom
THEN ERROR-PROCESSING STATEMENTS;
END;
```



déclancher

The diagram consists of a horizontal line starting from the word 'déclancher' on the right, extending to the left, and then turning downwards into an arrow pointing to the 'RAISE eExceptionNom;' statement in the code block.

Gestion des exceptions

Exemple:

```
DECLARE
```

```
    vTotalCourses NUMBER;
```

```
    eInvalidId EXCEPTION;
```

```
BEGIN
```

```
    IF vStudentId < 0 THEN
```

```
        RAISE eInvalidId;
```

```
    ELSE
```

```
        SELECT COUNT(*)
```

```
        INTO vTotalCourses
```

```
        FROM enrollment WHERE studentId = vStudentId;
```

```
        DBMS_OUTPUT.PUT_LINE
```

```
        ('L''étudiant est inscrit à '||vTotalCourses||' cours');
```

```
    END IF;
```

```
    DBMS_OUTPUT.PUT_LINE('Aucune exception n''a été levée');
```

```
EXCEPTION
```

```
    When eInvalidId THEN
```

```
        DBMS_OUTPUT.PUT_LINE('Un Id ne peut être négatif');
```

```
END;
```

Gestion des exceptions

```
DBMS_OUTPUT.PUT_LINE
('L''étudiant est inscrit à '||vTotalCourses||' cours');
END IF;
DBMS_OUTPUT.PUT_LINE('Aucune exception n''a été levée');
EXCEPTION
When eInvalidId THEN
DBMS_OUTPUT.PUT_LINE('Un Id ne peut être négatif');
END;
```

Curseur

- „ Un SELECT INTO peut être utilisé si la requête ne retourne qu'un enregistrement,
- „ Pour gérer les requêtes qui retournent un nombre Arbitraire d'enregistrement PL/SQL fournit les **curseurs**,
- „ Un **curseur** permet de récupérer tous les enregistrements mais d'accéder à un à la fois,
- „ Le **curseur** peut avancer de 1 pour accéder à l'enregistrement suivant,
- „ Un **curseur** doit être déclaré et ouvert avant d'être utilisé.

Curseur

- „ Il doit être fermé après son utilisation,
- „ Une fois ouvert, les enregistrements peuvent être accédés un à la fois en utilisant la commande **FETCH**,
- „ Le curseur doit être défini dans la section **DECLARE**,
- „ Il doit être ouvert dans la section d'exécution,
- „ Il est parcouru ensuite dans une boucle à l'aide du **fetch into**,
- „ On vérifie la fin du curseur avec **nomDuCurseur%NOTFOUND**,
- „ Il est fermé après la boucle.

Exemple de curseur

DECLARE

vNumPropriété PropriétéALouer.numPropriété%TYPE;

vRue PropriétéALouer.rue%TYPE;

vVille PropriétéALouer.ville%TYPE;

vCodePostal PropriétéALouer.codePostal%TYPE;

CURSOR propriétéCurseur **IS**

SELECT numPropriété, rue, ville, codePostal

FROM PropriétéALouer

WHERE numPersonnel = 'EM14'

ORDER BY numPropriété;

Exemple de curseur

```
BEGIN
-- Ouvrir le curseur au début de la sélection,
-- puis boucler pour prendre chaque ligne
-- de la table résultante.
OPEN propriétéCurseur;
LOOP
-- Prendre la ligne suivante de la table résultante.
FETCH propriétéCurseur
INTO vNumPropriété, vRue, vVille, vCodePostal;
EXIT WHEN propriétéCurseur%NOTFOUND;
-- Afficher les données.
dbms_output.put_line('N° de propriété : ' ||
vNumPropriété);
```

Exemple de curseur

```
dbms_output.put_line('Rue : ' || vRue);  
dbms_output.put_line('Ville : ' || vVille);  
IF codePostal IS NOT NULL THEN  
dbms_output.put_line('code postal : ' ||  
    vcodePostal);  
ELSE  
dbms_output.put_line('code postal : NULL');  
END IF;  
END LOOP;  
IF propriétéCurseur%ISOPEN THEN CLOSE  
    propriétéCurseur END IF;
```


Exemple de curseur

```
-- Si condition d'erreur Alors afficher l'erreur.  
EXCEPTION  
WHEN OTHERS THEN  
  dbms_output.put_line('Erreur détectée. ');  
  IF propriétéCurseur%ISOPEN THEN CLOSE  
    propriétéCurseur END IF;  
END;
```

Procédures et Fonctions

- „ Création de procédures et fonctions. Code enregistrable et réutilisable dans le SGBD.
- „ Procédure : traitement (pas de retour de résultat).
- „ Fonction : traitement (avec retour de résultat)

Procédures et Fonctions

```
CREATE [OR REPLACE ] PROCEDURE nom_procedure
    [(argument1...[,argumentN])] IS
    [Section de déclaration de variable]
BEGIN
    Section exécutable
    [Section_gestion_exception]
END [nom_procedure] ;
/
```

„ Où **argument1** à **argumentN** est sous la forme:

nom_argument [IN|OUT] type_données[:=|DEFAULT] valeur

„ Mettre un **/** à la fin pour réellement **exécuter** la commande,

„ Pour **exécuter** une procédure:

set serveroutput on;

execute nom_procedure(...);

Procédures et Fonctions

```
CREATE PROCEDURE spSalutLeMonde IS
    vSalutation varchar2(20);
BEGIN
    vSalutation := 'Salut le monde';
    dbms_output.put_line(vSalutation);
END spSalutLeMonde;
/
```

Procédures et Fonctions

```
CREATE [OR REPLACE] Function nom_fonction
[ (argument1...[, argumentN) ]
    RETURN type_données_fonction IS
    [ Section de déclaration de variable ]
BEGIN
    Section exécutable
RETURN valeur_fonction;
    [ Section_gestion_exception ]
End nom_fonction;
/
```

Procédures et Fonctions

```
CREATE OR REPLACE FUNCTION fBalance (pNoCompte  
    NUMBER) RETURN NUMBER  
IS  
    vBalanceCompte NUMBER(8,2) ;  
BEGIN  
    SELECT BalanceCompte INTO vBalanceCompte FROM  
        Compte WHERE noCompte = pNoCompte;  
    RETURN vBalanceCompte;  
END fBalance;  
/
```

Les déclencheurs

- „ Déclencheur ou trigger: block PL/SQL sauvegardé dans la base de données,
- „ Il est exécuté implicitement si un événement particulier arrive,
- „ Un événement : insert, delete ou update,
- „ Implanter des règles de gestion non définies par les Contraintes d'Intégrité Référentielles,
- „ Maintenir des règles de sécurité,
- „ Générer les valeurs des champs qui se calculent,
- „ Prévenir des accès incorrects.

Les déclencheurs

```
CREATE [OR REPLACE] TRIGGER nom_trigger  
    {BEFORE|AFTER} Evenement_déclenchant ON nom_table  
    [FOR EACH ROW] [WHEN condition]  
Bloc_pl/sql  
END;  
/
```

„ Où *Evenement_déclenchant* est l'un ou plusieurs des éléments suivants: INSERT, UPDATE ou DELETE.

Les déclencheurs

„ Les Triggers Before:

... Exécuter avant l'action BD (insert, update, delete),

... Utilisé quand il y a des champs à calculer ou
certaines vérifications à effectuer sur les champs,

„ Les triggers After:

... Exécuté après une action BD,

... Si les actions à effectuer ne sont pas spécifiées dans
un trigger Before,

```
CREATE OR REPLACE TRIGGER Enseignant_AUD  
AFTER UPDATE OR DELETE ON Enseignant
```

Les déclencheurs

```
CREATE OR REPLACE TRIGGER Enseignant_AUD  
AFTER UPDATE OR DELETE ON Enseignant  
DECLARE
```

```
    vType VARCHAR2(10);
```

```
BEGIN
```

```
    IF UPDATING THEN
```

```
        vType := 'UPDATE';
```

```
    ELSIF DELETING
```

```
    THEN
```

```
        vType := 'DELETE';
```

```
    END IF;
```

Évalué à vrai si mise à jour.



Évalué à vrai si suppression.



Les déclencheurs

```
UPDATE Statistique
SET UsagerTransaction = USER,
DateTransaction = SYSDATE
WHERE NomTable = 'Enseignant'
AND NomTransaction = vType;
IF SQL%NOTFOUND
THEN
    INSERT INTO Statistique
        VALUES ('Enseignant', vType, USER,
SYSDATE);
    END IF;
END;
/
```

Si la mise à jour n'a pas été effectuée

Les déclencheurs(Exemple 1)

```
CREATE OR REPLACE TRIGGER tEmpruntAIU AFTER INSERT
OR UPDATE OF dateRet ON Emprunt DECLARE
vNbMaxEmprunte number;
vNBMaxExemplaire number;
BEGIN
/* Chercher la quantité maximale des livres
   empruntés par l'adhérent */
SELECT MAX(COUNT(*)) INTO vNbMaxEmprunte FROM
   Emprunt
WHERE dateRet IS NULL GROUP BY noAdh;
/* S'il existe un adhérent qui emprunte plus de 5
   livres, l'insertion sera interdite */
```

Les déclencheurs(exemple 1, suite)

```
IF vNbMaxEmprunte > 5 THEN
    RAISE_APPLICATION_ERROR(-20056, 'On ne peut
    emprunter plus de '
    || '5 livres');
END IF;
/* Chercher la quantité maximale des exemplaires
   des oeuvres empruntés par l'adhérent */
SELECT MAX(COUNT(*)) INTO vNBMaxExemplaire FROM
    emprunt, Livre
WHERE (emprunt.cote=Livre.cote) AND (dateRet IS
    NULL)
GROUP BY noAdh, titre;
```

Les déclencheurs(exemple 1, suite)

```
/* S'il existe un titre qui est emprunté deux fois,  
   l'insertion est interdite */  
IF vNBMaxExemplaire > 1 THEN  
    RAISE_APPLICATION_ERROR(-20057,'Ce titre est déjà  
    emprunté');  
END IF;  
END;  
/
```

Les déclencheurs(exemple 2)

```
create or replace trigger TRG_AU_LIVRE_EMPRUNTE
after update on LIVRE
for each row
```

```
declare
```

```
    V_NB_EMPRUNT number;
```

```
begin
```

```
/* Vérifier si ce Livre est déjà emprunté */
```

```
    select count(*) into V_NB_EMPRUNT
```

```
        from EMPRUNT
```

Référence à l'ancien
valeur de la colonne

```
        where COTE = :OLD.COTE;
```

```
/* Si oui, refuser la MAJ*/
```

```
if V_NB_EMPRUNT >0 then
```

```
    raise_application_error(-20052, 'Ce Livre a été emprunté. La'
    || 'MAJ est interdite');
```

```
end if;
```

```
end;
```

```
/
```

Les déclencheurs

- „ **:OLD.<colonne>** : Référence à l'ancienne valeur de la colonne (valeur avant d'être modifiée),
- „ **:new.<colonne>** : référence à la nouvelle valeur de la colonne (valeur après avoir été modifiée),
- „ Disponibles seulement sur les déclencheurs définis pour Chaque rangées,
 - ... Toutefois, on ne peut pour ce genre de déclencheurs lire les autres enregistrements de la même table,
- „ Les **new** et **old** peuvent être utilisés avec les triggers comme suit:
 - ... **UPDATE:**
 - :**new**.<colonne>, :**old**.<colonne>
 - ... **INSERT:**
 - :**new**.<colonne>
 - ... **DELETE: :**
 - old**.<colonne> .

Les déclencheurs(exemple 3)

```
CREATE OR REPLACE TRIGGER tEnseignantBI
BEFORE INSERT ON Enseignant
FOR EACH ROW
DECLARE
vCodePostal CHAR(1);
BEGIN
:NEW.CreePar := USER;
:NEW.DateCreation := SYSDATE;
:NEW.ModifiePar := USER;
:NEW.DateModification := SYSDATE;
END;
/
```

Objectif-4'

Amélioration des performances

Objectifs -4'

- „ La dénormalisation
- „ Quand dénormaliser ?
- „ Surveiller et ajuster le système

Introduction de la redondance contrôlée.

- „ Se fait après la mise en exploitation
- „ peut s'avérer utile après l'analyse des données.
- „ peut améliorer la recherche

Attention

Rend l'implémentation complexe

Sacrifie souvent la souplesse

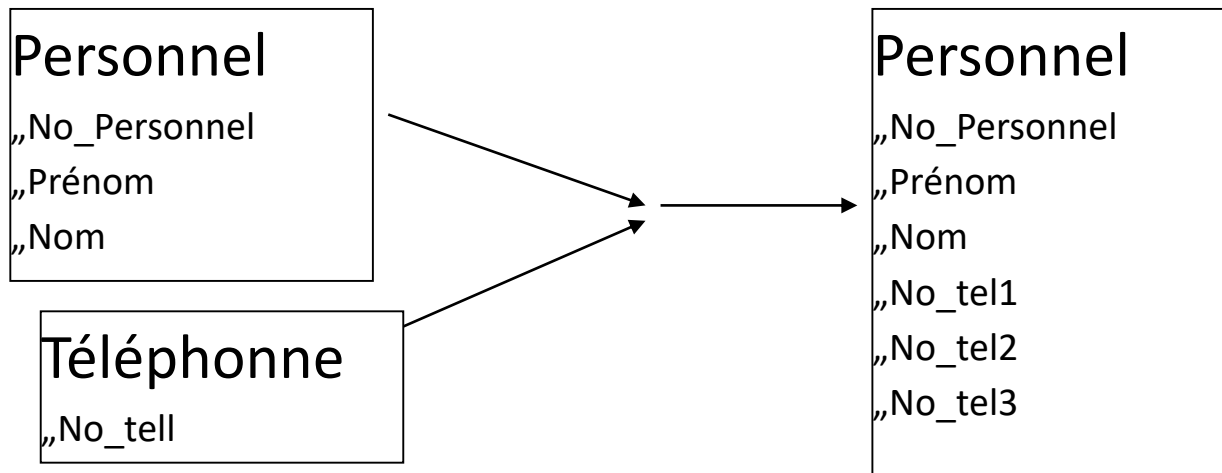
Peut ralentir la mise à jour

Introduction de la redondance contrôlée.

L'on peut :

- „ fusionner les tables ayant un lien 1:1
- „ fusionner les tables ayant un lien n:n en dupliquant un attribut ➔ introduction des groupes répétitifs
- „ Etc.

Exemple



Création de table d'extraction.

„ Consiste à créer des tables pour faciliter la production des rapports.

Partitionner les relations.

„ Horizontal

„ Verticale

Après la dénormalisation.

„ Réétudier le maintien des index et des contraintes d'intégrité.

„ Revoir les déclencheurs (triggers)

La dénormalisation est susceptible
d'améliorer la performance par :

- „ Moins de calcul
- „ Moins de jointures
- „ Moins de clés étrangères dans les relations
- „ Moins d'index → réduction d'espace de stockage
- „ La réduction du nombre de relations

Inconvénients:

La dénormalisation est susceptible de :

- „ ralentir les Mise à jours
- „ être très spécifique à une application.
- „ accroître la taille des relations
- „ compliquer l'implantation

Surveiller et régler le système :

„ Corriger les décisions inadéquates

„ Facteur de d'efficacité :

- Le taux (débit) de transaction
- Le temps de réponse
- L'espace de stockage

„ Avantages :

- Matériel
- Sur les utilisateurs

„ Surveillez :

- La mémoire principale
- L'unité centrale
- Entrées/sorties disques (séparer les disques)
- Le réseau(collisions)
- Documentation

Objectif 5'

TRAITEMENT DES REQUÊTES

Chapitre 10' - Objectifs

- Traitement et optimisation des requêtes.
- L'optimisation statiques et dynamique
- Manière de décomposer et d'analyser une requêtes sur le plan sémantique
- Construction d'un arbre en algèbre relationnelle pour répondre aux requêtes
- La façon dont Oracle gère les optimisations

Introduction

- Optimiser
 - pour exécuter rapidement
 - pour consommer moins de ressources
- Peut se faire
 - par le programmeurs
 - ou par le SGBD
- Deux techniques principales :
 - Règles d'optimisation
 - Comparaison de différentes stratégies basées sur le coût
- L'accès disque est l'opération la plus coûteuse

Traitement de requêtes (TR):

Les activités qui impliquent le balayage, le contrôle de validité, et l'optimisation et l'exécution d'une requête.

Le but du TR

- Transformer les requêtes écrites dans un langage de haut niveau (SQL) en une stratégie d'exécution correcte et efficace exprimée dans un langage de bas niveau.
- Exécuter la stratégie pour obtenir les données demandées.

Optimisation de requêtes (OR) :

- Les activités impliquées par le choix d'une stratégie d'exécution correcte pour exécuter une requête.
- Puisqu'il peut y avoir plusieurs transformations équivalentes de la même requête de même niveau, le but de l'OR est de choisir celle qui réduit l'usage des ressources.
- Réduit le temps d'exécution total de la requête ou le temps de réponse,
- Pour un problème insoluble, lorsqu'un grand nombre de relations intervient, la stratégie adoptée est de trouver une solution proche de l'optimum.

Exemple:

Trouver tous les gérants qui travaillent dans une filiale de Paris.

```
SELECT *  
FROM Personnel p, Filiale f  
WHERE p.numFiliale = f.numFiliale  
AND p.fonction = 'Gérant' AND f.ville='Paris' ;
```


Exemple : Différentes stratégies :

3 requêtes algébriques équivalentes : 5 succursales à paris, aucun index

$$1. \sigma_{\text{(fonction='Gérant')} \wedge \text{(ville='Paris')} \wedge \text{(Personnel.numFiliale = Filiale.numFiliale)}} (\text{Personnel X Filiale})$$

$$2. \sigma_{\text{(fonction='Gérant')} \wedge \text{(ville='Paris')}} (\text{Personnel} \bowtie_{\text{(Personnel.numFiliale = Filiale.numFiliale)}} \text{Filiale})$$

$$3. (\sigma_{\text{fonction='Gérant'}} (\text{Personnel})) \bowtie_{\text{Personnel.numFiliale = Filiale.numFiliale}} (\sigma_{\text{ville='Paris'}} (\text{Filiale}))$$

Exemple : Différentes stratégies :

On suppose que Personnel contient 1000 enregistrements, 50 tuples dans Filiale, on a 50 gérants
5 succursales à Paris

Exemple : Comparaison de coût

Les coût en accès disque sont :

(1)

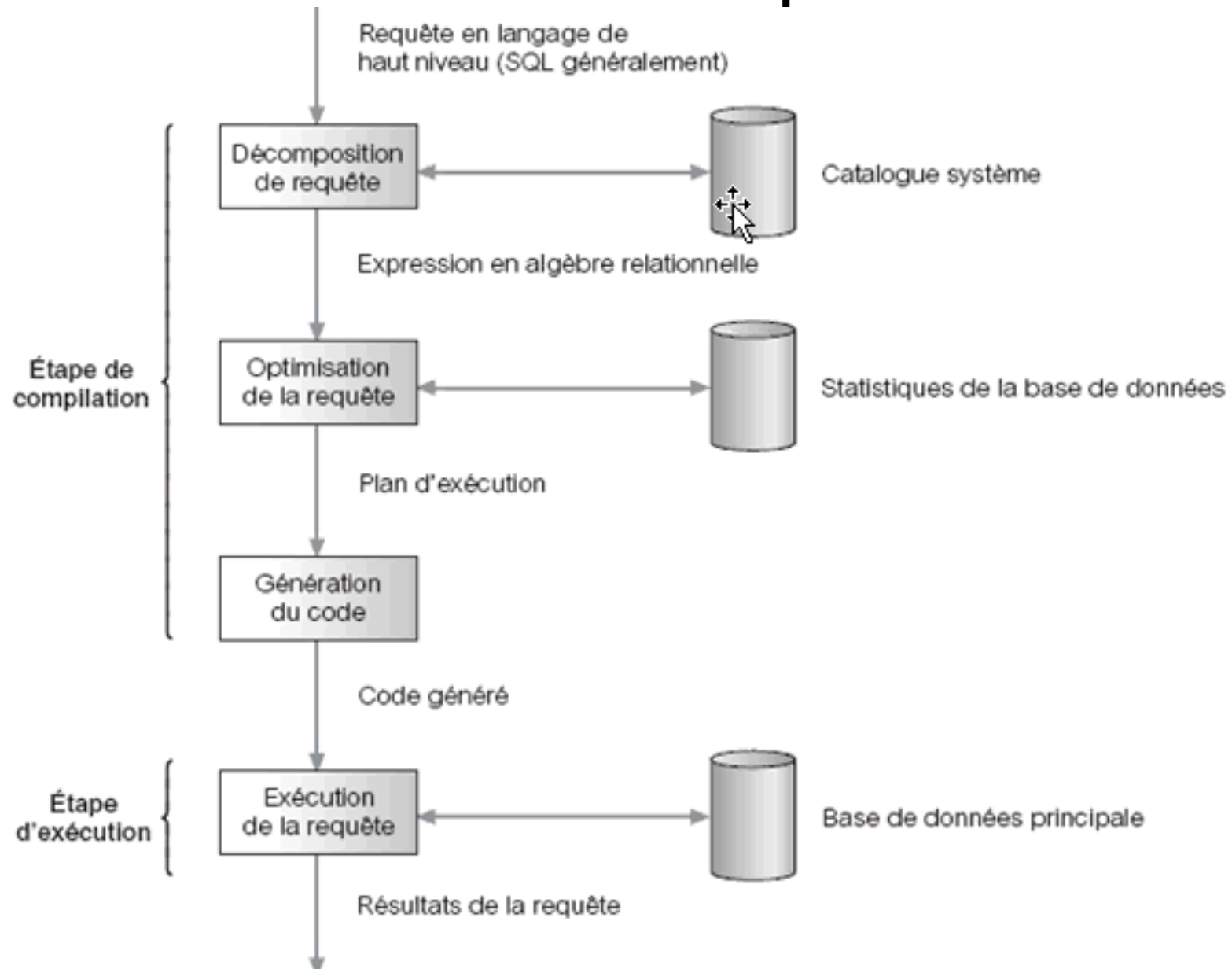
- Produit cartésien de Personnel et Filiale ($1000 + 50$) pour lire les relations,
- Créer une relation de $(1000*50)$ tuples,
- Lire une nouvelle fois pour tester le prédicat de sélection $(1000*50)$ soit **101 050**

(2) $2*1000+(1000+50) = \mathbf{3\ 050}$

(3) $1000+2*50+5+(50+5)=\mathbf{1\ 160}$

• Les opérations de produit cartésien et de jointure sont plus coûteuses que les sélection tandis que la troisième option réduit de façon significative la grandeur des relations jointes.

Phases de traitement de requête



Optimisation dynamique vs Statique

- Lorsque les 3 premières phases peuvent s'effectuer de façon :
 - a) Statique lorsque la requête est exécutée pour la première fois
 - b) Dynamique à chaque exécution de la requête.
- L'avantage de la dynamique apparaît lorsque l'information est mise à jour.
- Le désavantage est que les performances de la requête sont affectées
- Le temps peut limiter la recherche de stratégies optimale.

Optimisation dynamique vs Statique

- L'avantage de l'optimisation statique est l'absence de surcharge à l'exécution et donc plus de temps pour la recherche de stratégie optimale
- Le désavantage apparaît du fait que la stratégie d'exécution choisie peut ne plus être optimale lorsque la requête est exécutée.
- On peut utiliser une approche hybride pour compenser.

Décomposition de requête

- Transformer une requête en langage de haut niveau en une requête algébrique et vérifier que la requête algébrique est syntaxiquement et sémantiquement correcte
- Les étapes sont :
 - ✓ Analyse
 - ✓ Normalisation
 - ✓ Analyse sémantique
 - ✓ Simplification
 - ✓ Restructuration de la requête

Analyse

- La requête est analysée sur le plan lexical et syntaxique à l'aide des techniques des compilateurs de langage de programmation.

Analyse : exemple

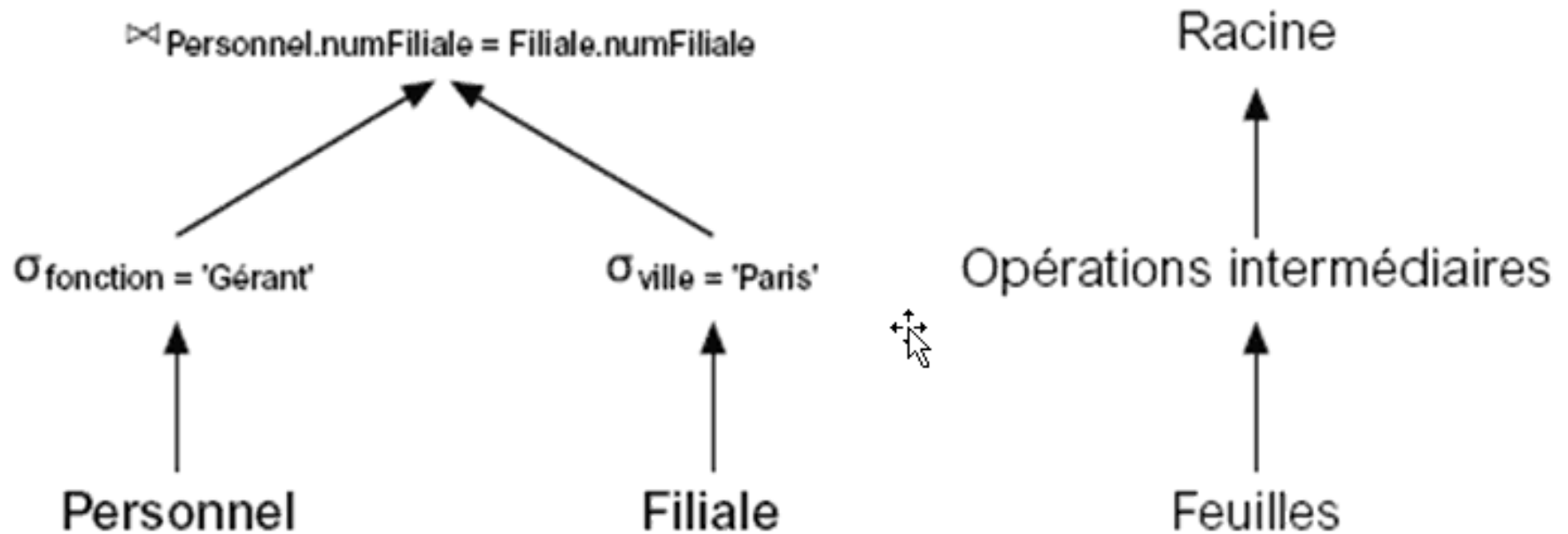
```
SELECT numéroPersonnel  
FROM Personnel  
WHERE fonction >10 ;
```

- Cette requête est rejetée pour deux raisons
- Dans la liste select : NuméroPersonnel n'existe pas dans Personnel;
- Dans la clause where : Fonction est de type chaîne de caractères;

Analyse : exemple

- Finalement la requête est transformée en une représentation interne mieux adaptée au traitement.
- Un **arbre algébrique** est généralement choisi et construit comme suit.
- Un nœud feuille est créé pour chaque relation de base.
- Un nœud est créé pour chaque relation intermédiaire produit par l'opération algébrique
- La racine représente le résultat de la requête.
- La séquence est dirigée des feuilles à la racine.

Analyse : arbre relationnel



Normalisation

- Convertir la requête en forme normale plus facile à manipuler
- Le prédicat peut être convertie en une de ces formes.
- Forme normale conjonctive :

$(\text{fonction} = \text{'Gérant'} \vee \text{salaire} = 20000) \wedge \text{numFiliale} = \text{'F003'}$

- Forme normale disjonctive

$(\text{fonction} = \text{'Gérant'} \wedge \text{numFiliale} = \text{'F003'}) \vee (\text{salaire} = 20000 \wedge \text{numFiliale} = \text{'F003'})$

Analyse sémantique

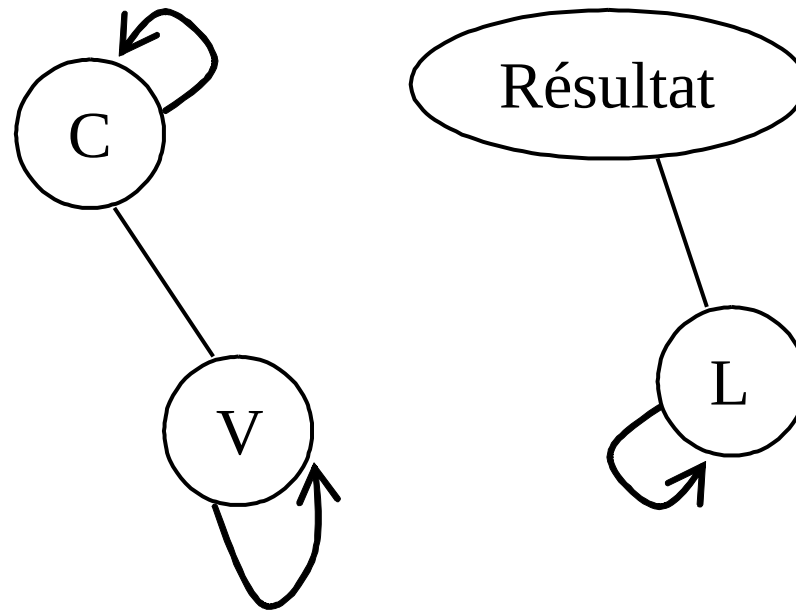
- Rejeter les requêtes qui sont **incorrectement** formulé ou **contradictaires**
- Une requête est **incorrectement** formulé si ses composantes ne contribuent pas à générer le résultat.
- Une requête est **contradictoire** si son prédicat ne peut être satisfait par aucun tuple.
- Pour analyser, on peut construire :
 - Un graphe de connexion de relations
 - Un graphe normalisé de connexion d'attributs

Graphe de connexion de relations

- Créer un nœud pour chaque relation et un nœud pour le résultat.
- Créer les liens entre les nœuds qui représentent une jointure et les liens entre les nœuds qui représentent une projection
- S'il y a des parties non connectée la requête est incorrecte.

Exemple de validation de rectitude sémantique

- SELECT l.numPropriété, l.rue
FROM Client c, Ville v, PropriétéALouer l
WHERE c.numClient = v.numClient
AND c.locMax>=500
AND c.typePréf = 'Appartement'
AND L.numPropriété='CP93'



Graphe de connexion de relations

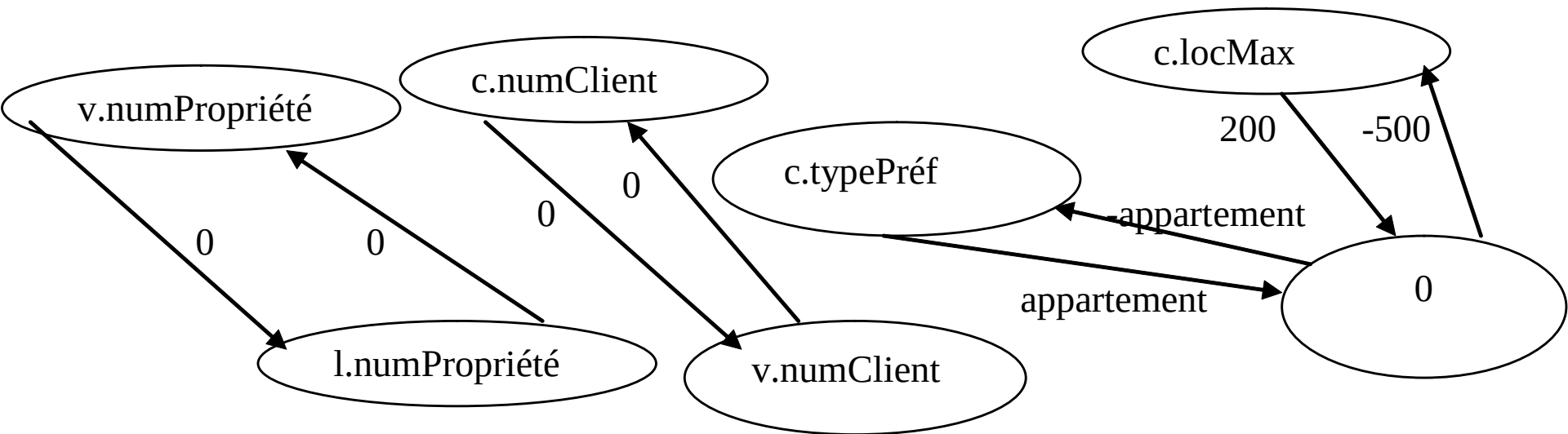
- Le graphe de connexion de relation n'est pas complètement connecté, la requête n'est pas formulée correctement.
- La condition de jointure a été omise.
- $v.\text{numPropriété} = l.\text{numPropriété}$

Le graphe normalisé de connexion d'attributs

- Créer un nœud pour chaque référence à un attribut ou une constante 0
- Créer une flèche entre les nœuds qui représentent une jointure et entre les nœuds attributs et le nœud 0 qui représente la restriction
- Pondérer la flèche $a \rightarrow b$ par c si elle représente une condition $(a \leq b + c)$
- Pondérer la flèche $0 \rightarrow a$ par $-c$ si elle représente une condition $(a \geq c)$
- Si le graphe contient des cycles dont la somme est négative, la requête est contradictoire.

Exemple de validation de rectitude sémantique

- SELECT l.numPropriété, l.rue
FROM Client c, Ville v, PropriétéALouer l
WHERE c.numClient = v.numClient
AND v.numPropriété=l.numPropriété
AND c.locMax>=500
AND c.typePréf = 'Appartement'
AND c.locMax<200;



Simplification

- Détecter les qualifications redondantes,
- Élimination des sous expressions communes
- Transformer la requête en un équivalent sémantique mais sous une forme plus facile à calculer et plus efficace,
- Sont considérés :
 - Les restrictions d'accès,
 - Les définitions de vues,
 - Les contraintes d'intégrité
- En supposant que les utilisateurs ont les privilèges d'accès appropriés, applique d'abord les règles d'idempotence d'algèbre booléenne
- .

Optimisation de requêtes sous oracle

- Oracle supporte deux approches d'optimisation de requêtes.
- Orienté règles
- Orienté coût

Optimisation oracle : Orienté règles

- 15 règles classées en ordre d'efficacité
- Un chemin particulier pour une table est choisi seulement si la requête contient un prédicat qui rend ce chemin accessible
- Un pointage est assigné à chaque stratégie d'exécution en utilisant ce classement
- La stratégie avec le meilleur (le faible) classement est choisie
- Lorsque deux stratégies produisent le même pointage, le choix est fait en fonction de l'ordre d'apparition des tables dans la requête.

Optimisation oracle : Orienté règles

Classement	Méthode d'accès
1	Une seule ligne par ROWID (identificateur de ligne)
2	Une seule ligne par jointure de cluster
3	Une seule ligne par clé de cluster haché avec clé unique ou primaire
4	Une seule ligne par clé unique ou primaire
5	Jointure de cluster
6	Clé de cluster haché
7	Clé de cluster indexé
8	Clé composite
9	Index sur des colonnes uniques
10	Recherche sur une plage délimitée dans des colonnes indexées
11	Recherche sur une plage non délimitée dans des colonnes indexées
12	Jointure par tri interclassement
13	MAX ou MIN sur des colonnes indexées
14	ORDER BY sur des colonnes indexées
15	Parcours de la totalité de la table

Optimisation oracle : Orienté règles

```
SELECT numPropriété  
FROM PropriétéALouer  
Where pièces > 7 AND ville = 'Paris'
```

- Chemin d'accès de colonne simple utilisant un index sur ville à partir de la condition WHERE ville='Paris' : 9
- Parcours sans limite utilisant l'index sur pièces à partir de la condition WHERE pièces>7 : 11
- Parcours complet de la table : 15
- Bien qu'il y ait un index sur numPropriété, la colonne n'est pas utilisée dans la clause WHERE et n'est donc pas considéré.
- Basée sur les chemins , l'optimiseur orienté règles choisi d'utiliser l'index de la colonne ville

Optimisation oracle : Orienté coût

- Depuis Oracle 7 : Choisit la stratégie qui requière un minimum de ressources utilisées nécessaires pour traiter toutes les rangées accédées par la requête.
- L'utilisateur peut sélectionner si l'utilisation minimale des ressources est basée sur ce qui est retourné ou sur le paramètre d'initialisation OPTIMIZER_MODE,
- L'optimiseur basée coût prend aussi les indices que l'utilisateur peut fournir

Optimisation oracle - statique

- L'optimiseur orienté coût dépend des statistiques de toutes les tables, cluster, et index accédés par la requête.
- C'est à l'utilisateur de générer ces statistiques et de les garder à jour.
- Le package DBMS_STATS peut être utilisé pour générer et gérer les statistiques
- **EXECUTE DBMS_STATS.Gather_Schema_Stats('Manager')**

Optimisation oracle - Histogramme

- Une supposition est faite que les données sont uniformément distribuées dans une colonne
- L'histogramme des valeurs et leur fréquences relative donne à l'optimiseur une estimation améliorée de la sélection en présence de distribution non-uniforme

Optimisation oracle - Histogramme

- L'histogramme est une structure de données qui peut améliorer l'estimation du nombre de tuples d'un résultat.
- Deux types d'histogrammes:
 - Histogramme à largeur balancée qui divise les données en nombre fixe d'intervalle (appelées boîtes) chacune contenant un nombre de valeurs tombant dans l'intervalle
 - Histogramme à hauteur balancée qui place approximativement le même nombre de valeur dans chaque boîte

Visualiser le plan d'exécution

- Se fait avec la commande
- EXPLAIN PLAN
- kisito@yahoo.com kisito@univ-ouaga.bf

```
SQL> EXPLAIN PLAN
  2 SET STATEMENT_ID = 'PB'
  3 FOR SELECT b.branchNo, b.city, propertyNo
  4 FROM Branch b, PropertyForRent p
  5 WHERE b.branchNo = p.branchNo
  6 ORDER BY b.city;

Explained.

SQL> SELECT ID||' '||PARENT_ID||' '||LPAD(' ', 2*(LEVEL - 1))||OPERATION||' '||OPTIONS||
  2 ' '||OBJECT_NAME "Query Plan"
  3 FROM Plan_Table
  4 START WITH ID = 0 AND STATEMENT_ID = 'PB'
  5 CONNECT BY PRIOR ID = PARENT_ID AND STATEMENT_ID = 'PB';

Query Plan
-----
0      SELECT STATEMENT
1 0      SORT ORDER BY
2 1      NESTED LOOPS
3 2          TABLE ACCESS FULL PROPERTYFORRENT
4 2          TABLE ACCESS BY INDEX ROWID BRANCH
5 4              INDEX UNIQUE SCAN SYS_C007455

6 rows selected.
```

Objectif 6'

SÉCURITÉ DES BASES DE DONNÉES

Sécurisation d'une BD : objectifs

”

• Assurer la sécurité d'une BD c'est maintenir :

- la **confidentialité** des données,
- **l'intégrité** des données,
- et la **disponibilité** des données

•

”

Maintien de la confidentialité

Il s'agit de détecter ou d'empêcher des accès non autorisés.

C'est crucial :

- dans des environnements critiques ou stratégiques : militaires ou commerciaux, par exemple.
- pour respecter le droit des individus à décider comment et dans quel but les informations les concernant peuvent être extraites, mémorisées ou transmises à d'autres individus.

Maintien de l'intégrité

Il s'agit de détecter ou d'empêcher des **modifications illicites** des données qu'elles soient dues à :

- des **pannes** de système,
- des **manipulations** erronées,
- des **sabotages**

Maintien de la disponibilité

Il s'agit de détecter ou d'empêcher des **dénis de service**.

Il y a déni de service lorsqu'un utilisateur ne parvient pas à accéder dans un **délai raisonnable**, à une information ou à une ressource pour laquelle il a une autorisation d'accès.

Par exemple, une attaque consistant à **saturer** un serveur de **fausses requêtes** empêchant les requêtes valides d'être exécutées.

Problème : Les Attaques

- Les **violations** de la sécurité d'une BD consistent en des **lectures** ou des **prises à jour illicites**.
- Les événements qui portent ces violations sont appelées des **attaques**

Solutions :

- Authentification
- Contrôle d'accès
- Politique de sécurité
- Principe du moindre privilège
- Autorisation, interdiction et obligation
- Modèles de contrôle d'accès
- Sécurisation des applications
- Audit

Modèles à base de rôles

C'est le « **Modèle de contrôle d'accès** » le plus utilisé.

On peut améliorer la discrétion en créant des **rôles** qui sont des ensembles d'autorisation.

- Les autorisations sont octroyés aux rôles et les rôles aux utilisateurs.
- Pour pouvoir réaliser une action sur un objet un utilisateur doit ouvrir une session et activer celui de ses rôles qui contient l'autorisation de réaliser cette action.
- Un rôle peut hériter des privilèges d'un autre rôle.

Contrôle d'accès en SQL

Contrôle d'accès en SQL

Sujets

utilisateurs, groupe d'utilisateurs, tous les utilisateurs

Objets

BD, tables, vues, index, procédures...

Privilèges (ou autorisations)

sur les tables,
sur le schéma,
sur la base de données...



Rôles

groupes de privilèges

Contrôle d'accès en SQL :

Exemple : cas d'Oracle

Création d'un utilisateur

```
CREATE USER user_name IDENTIFIED BY password;
```

Connections

```
CONNECT user_name / password@Nom_De_La_BaseDeDonnées ;
```

Changer son mot de passe

```
ALTER USER user_name IDENTIFIED BY new_password;
```

Octroi ou révocation de privilèges

SQL offre deux commandes pour octroyer ou révoquer des privilèges :

1. GRANT

2. REVOKE

GRANT

Syntaxe

GRANT (*liste de privilèges* | **ALL**)
ON *liste d'objets*
TO *liste d'utilisateurs* | **PUBLIC**
[WITH GRANT OPTION]

avec :

ALL

tous les privilèges que le donneur peut accorder

PUBLIC

tous les utilisateurs connus du système

WITH GRANT OPTION

indique que le receveur pourra transmettre les
privilèges qui lui sont octroyés

Syntaxe

REVOKE [GRANT OPTION FOR] (*privilèges* | ALL)
ON *liste d'objets*
FROM *liste d'utilisateurs*
[**{RESTRIC**T / **CASCADE**}]

avec :

CASCADE

la révocation concerne les utilisateurs cités dans la clause FROM ainsi que ceux à qui ces privilèges ont été récursivement transmis.

RESTRIC

la révocation ne concerne que les utilisateurs cités dans la clause FROM.

GRANT OPTION FOR

ce n'est pas les privilèges qui sont révoqués, mais le droit de le transmettre.

Rôles

- Un **rôle** est un ensemble de privilèges.
- Les rôles sont assignés aux utilisateurs qui peuvent avoir plusieurs rôles.
- Les rôles sont organisés hiérarchiquement.
-
- Il ne faut pas confondre rôle et utilisateur :
un utilisateur peut être propriétaire d'un objet, un rôle ne le peut pas

Gestion des rôles en SQL Oracle

Création d'un rôle

CREATE ROLE *nom de rôle*

Octroi d'un privilège à un rôle

GRANT *liste de privilèges*

ON *liste d'objets*

TO *liste de rôles ou d'utilisateurs*

Octroi d'un rôle à un rôle ou à un utilisateur

GRANT *liste de rôles*

TO *liste de rôles ou d'utilisateurs*

Activation d'un rôle

SET ROLE *liste de rôles*

Références :

- Livres de A.Gamache
- Support de M.P.Parent